КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА

ЛАБОРАТОРНА РОБОТА № 2

Пошук найкращого ходу в детермінованій грі Lines of Action із застосуванням Minimax та Alpha-beta-відтинання

Виконав: Венц Ю В
Група: ТТП42

Київ – 2026

1. Мета і завдання роботи

Мета роботи полягає у дослідженні алгоритму Minimax та оптимізації пошуку за допомогою Alpha-beta-відтинання на прикладі детермінованої гри з повною інформацією Lines of Action (LOA).

Для досягнення мети необхідно реалізувати правила гри, генератор допустимих ходів, функцію корисності завершених позицій, евристичну функцію оцінювання, пошук з обмеженням глибини, Alpha-beta-відтинання, упорядкування ходів та експериментальне порівняння алгоритмів.

2. Правила гри Lines of Action

Lines of Action є двоосібною детермінованою грою з повною інформацією на дошці 8×8. На початку чорні фігури розміщуються вздовж верхнього та нижнього країв без кутових клітин, а білі – вздовж лівого та правого країв. Першими ходять чорні.

Фігура рухається по горизонталі, вертикалі або діагоналі рівно на стільки клітин, скільки фігур обох кольорів розташовано на повній лінії руху. Дозволено перестрибувати через власні фігури, але заборонено перестрибувати через фігури суперника. Якщо кінцева клітинка зайнята фігурою суперника, вона знімається. Перемагає гравець, усі фігури якого утворили одну зв’язну компоненту за вісьмома напрямками.

Формалізація правил як системи станів

Компонент моделі	Формальне визначення
Поле	Матриця 8×8; клітинка належить множині {порожня, чорна, біла}.
Початковий стан	По 12 фігур кожного кольору вздовж протилежних країв без кутових клітин; першими ходять чорні.
Напрямок	Горизонталь, вертикаль або одна з двох діагоналей, у будь-який бік.
Відстань	Кількість усіх фігур обох кольорів на повній лінії руху.
Шлях	Через власні фігури перестрибувати можна; через фігури суперника не можна.
Захоплення	Фігура суперника на кінцевій клітинці знімається.
Виграш	Усі фігури гравця утворюють одну зв’язну компоненту за вісьмома напрямками.

Початкова позиція є симетричною, тому на малих глибинах кілька ходів можуть мати однакову оцінку. У такому разі конкретний вибір залежить від порядку обходу, але minimax-значення залишається однаковим.

3. Формальна модель гри

Стан гри задається кортежем S = (B, p, l, k), де B – матриця дошки 8×8; p – гравець, який має виконати хід; l – гравець, який виконав попередній хід; k – номер напівходу. Значення клітинки належить множині {0, 1, −1}: порожня клітинка, чорна або біла фігура.

Функція переходу result(S, a) створює новий стан після допустимого ходу a: очищує початкову клітинку, переносить фігуру в кінцеву клітинку, за потреби виконує захоплення та змінює активного гравця.

Пошукове дерево та рекурсивне значення

Вузол дерева відповідає повному стану гри, а ребро — одному допустимому ходу. Рівні MAX і MIN чергуються. Глибина d вимірюється у напівходах від поточної позиції, а не у повних раундах гри.

$$V(S,d) = UF(S), \text{ якщо } S \text{ термінальний}$$
$$V(S,0) = HEF(S), \text{ якщо гра триває}$$
$$V(S,d) = \max V(result(S,a),d-1), \text{ якщо хід MAX}$$
$$V(S,d) = \min V(result(S,a),d-1), \text{ якщо хід MIN}$$

Повний перебір має порядок O(b^d), де b — середня кількість допустимих ходів. У початковій позиції LOA програма згенерувала 36 ходів, тому навіть невелике збільшення глибини різко збільшує дерево.

4. Функції UF та HEF

4.1. Функція корисності

Для термінальних позицій використано точну функцію корисності: UF(S) = 1 000 000 − k у разі перемоги MAX та UF(S) = −1 000 000 + k у разі перемоги MIN. Поправка на номер напівходу надає перевагу швидшій перемозі та пізнішій поразці.

4.2. Евристична функція

Для позицій на межі глибини використано складену евристику:

$$HEF = 120\Delta L - 45\Delta C + 18\Delta P + 2\Delta M + 1{,}5\Delta Z + 3\Delta K$$

Тут L – розмір найбільшої зв’язної компоненти; C – кількість компонент; P – кількість фігур; M – кількість допустимих ходів; Z – центральність; K – компактність. Символ Δ означає різницю значення ознаки для MAX і MIN. Найбільшу вагу мають зв’язність та кількість компонент, оскільки вони безпосередньо відповідають меті LOA.

Позначення	Вага	Ознака	Обґрунтування
ΔL	120	Розмір найбільшої компоненти	Безпосередньо показує наближення до об’єднання всіх фігур.
ΔC	−45	Кількість компонент	Розрізнені групи ускладнюють

			досягнення мети LOA.
ΔP	18	Кількість фігур	Враховує матеріальний результат захоплень.
ΔM	2	Мобільність	Більша кількість ходів дає більше стратегічних можливостей.
ΔZ	1,5	Центральність	Центральні фігури зазвичай мають більше напрямків руху.
ΔK	3	Компактність	Менший розкид фігур полегшує їх об'єднання.

Ваги підібрано експертно: критерії зв'язності домінують над допоміжними позиційними ознаками. Значення HEF не є доведеною перемогою і застосовується лише до нетермінальних листків на межі пошуку.

5. Алгоритми пошуку

5.1. Minimax

Алгоритм Minimax повністю перебирає дерево до заданої глибини. У вузлах MAX обирається найбільше значення дочірніх станів, а у вузлах MIN – найменше. На межі глибини повертається HEF, а для завершених станів – UF.

5.2. Alpha-beta-відтинання

Alpha-beta є оптимізацією Minimax. Межа α зберігає найкраще відоме значення MAX, а β – найкраще відоме значення MIN. Якщо $\alpha \geq \beta$, подальший перегляд поточної гілки припиняється, оскільки вона не може змінити остаточний вибір. Оптимізація не змінює minimax-оцінку.

5.3. Упорядкування ходів

Перед рекурсивним пошуком ходи сортуються за швидкою позиційною оцінкою. Першими розглядаються переможні ходи та ходи, що зменшують кількість власних компонент. Якісне впорядкування раніше звужує межі α і β та збільшує число відтинань корисних піддерев.

6. Архітектура програмної реалізації

Компонент	Призначення
State	Зберігання дошки, активного гравця, генерація ходів і перевірка перемоги
Move	Координати початкової та кінцевої клітинок, шахова нотація ходу

evaluate	Обчислення UF або HEF
search	Minimax, Alpha-beta, обмеження глибини, статистика та журнал
App	Графічний інтерфейс Tkinter, запуск гри та експерименту

7. Експериментальне дослідження

Експеримент виконано з початкової позиції для глибин 1, 2 і 3. Для кожної глибини послідовно запускалися повний Minimax, Alpha-beta без сортування та Alpha-beta з попереднім сортуванням ходів. Усі варіанти використовували однакову UF/HEF.

Глиб.	Алгоритм	Хід	Оцінка	Вузли	Відт.	Час, с
1	Minimax	b8-h8	-2.2	37	0	0.1253
1	Alpha-beta	b8-h8	-2.2	37	0	0.0614
1	AB + сортування	b8-h8	-2.2	37	0	0.0499
2	Minimax	b8-h8	-15.8	1281	0	1.4419
2	Alpha-beta	b8-h8	-15.8	194	34	0.1773
2	AB + сортування	b8-h8	-15.8	113	35	0.2143
3	Minimax	b8-d6	34.1	46233	0	51.7744
3	Alpha-beta	b8-d6	34.1	3191	228	3.1316
3	AB + сортування	b8-d6	34.1	2624	186	3.5392

Таблиця 1 – Результати порівняльного експерименту

На всіх глибинах три варіанти повернули однакові хід та оцінку, що підтверджує еквівалентність результату Minimax та Alpha-beta. На глибині 1 відтинання відсутні, оскільки дерево містить лише корінь і листки. На глибині 2 Alpha-beta зменшив кількість вузлів з 1281 до 194, а сортування – до 113. На глибині 3 кількість вузлів зменшилася з 46 233 до 3 191 та 2 624 відповідно.

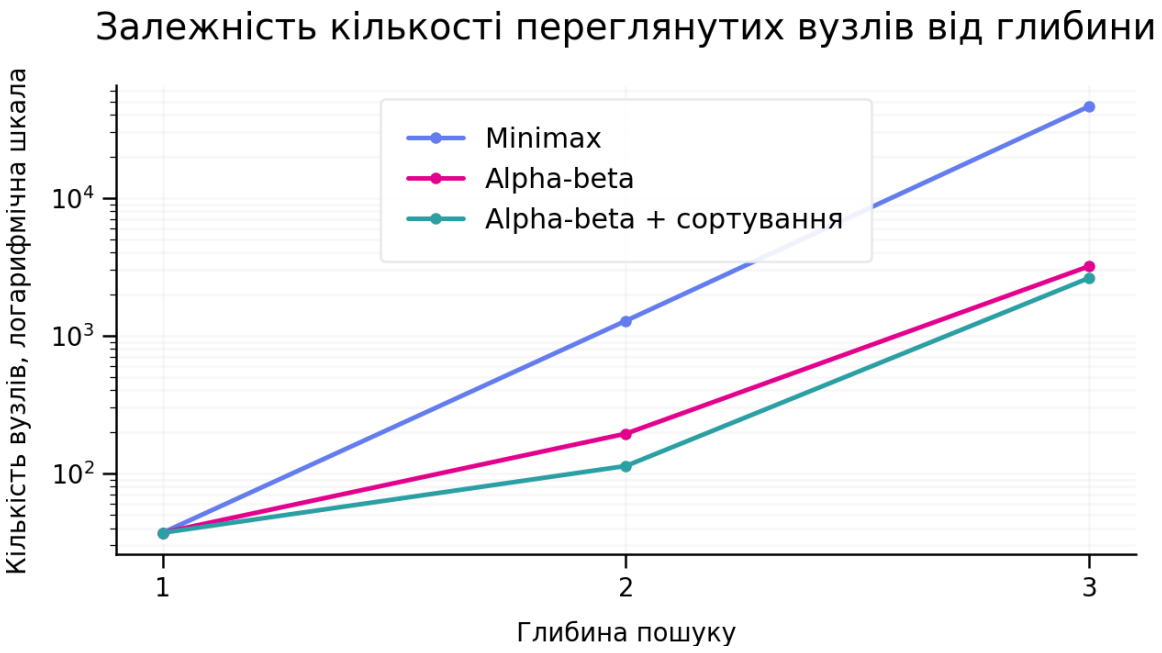


Рисунок 1 – Кількість переглянутих вузлів залежно від глибини

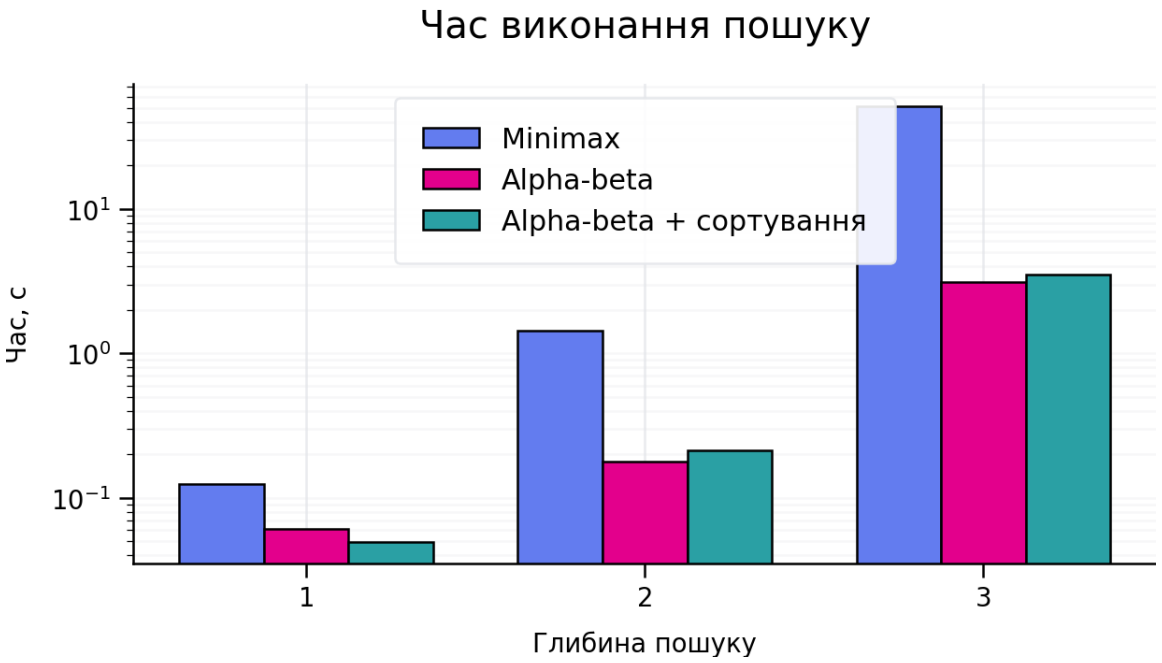


Рисунок 2 – Час виконання алгоритмів залежно від глибини

8. Аналіз ефективності

На глибині 3 Alpha-beta без сортування переглянув у 14,49 раза менше вузлів за Minimax: 46 233 / 3 191. Скорочення становить 93,10%. За часом прискорення становило 16,53 раза: 51,7744 / 3,1316.

Alpha-beta із сортуванням переглянув 2 624 вузли, тобто на 17,77% менше від Alpha-beta без сортування і на 94,32% менше від Minimax. Водночас час 3,5392 с виявився дещо більшим за 3,1316 с без сортування. Причиною є накладні витрати на створення дочірніх станів і швидке

оцінювання кожного ходу. Отже, менша кількість вузлів не гарантує меншого часу, якщо впорядкування саме по собі є дорогим.

9. Приклад Alpha-beta-відтинання

Журнал пошуку на глибині 3 містить такий фрагмент:

```
      MAX d=1, alpha=-inf, beta=13.3
h8-g7: v=11.8, alpha=11.8, beta=13.3
c8-a6: v=34.9, alpha=34.9, beta=13.3
ВІДТИНАННЯ: alpha >= beta; відкинуто 29 ходів
```

Після оцінювання ходу c8-a6 значення α стало 34,9, тоді як β дорівнювало 13,3. Оскільки 34,9 ≥ 13,3, решта 29 ходів у цьому MAX-вузлі не могла вплинути на рішення попереднього MIN-вузла та була відкинута.

10. Обмеження реалізації та евристики

Інтерфейс і результат виконання

На рисунку показано стан гри після кількох напівходів. У нижній частині вікна програма виводить хід комп’ютера h8-c8, евристичну оцінку 206,6, кількість переглянутих вузлів 170, число відтинань 31 і час пошуку 0,353 с за глибини 2.

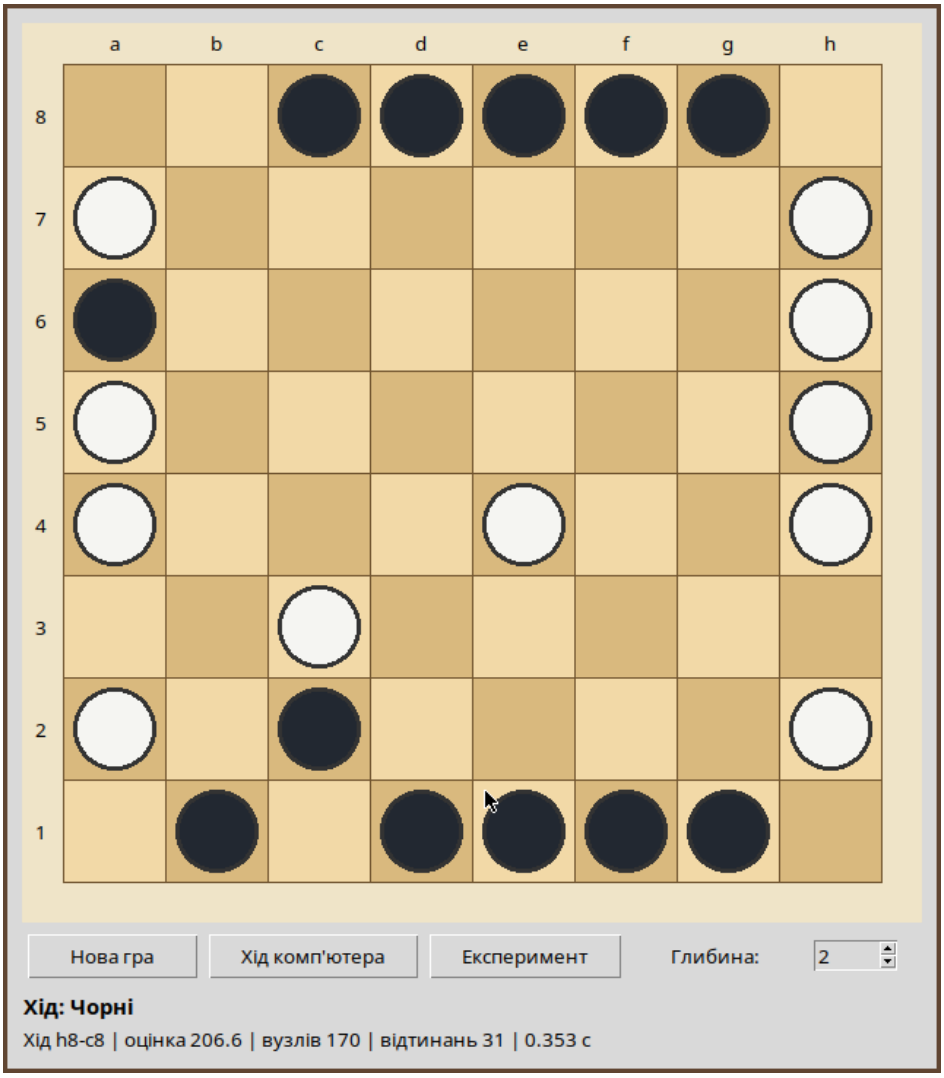


Рисунок 3 – Інтерфейс Lines of Action та статистика ходу Alpha-beta

Інтерфейс та спостережуваний результат

Графічний інтерфейс дає змогу розпочати нову гру, виконати хід комп’ютера, вибрати глибину та запустити порівняльний експеримент. Після ходу виводяться нотація, HEF/UF-оцінка, кількість вузлів, число відтинань і час. Експеримент не змінює поточну позицію, тому всі режими отримують однаковий стан.

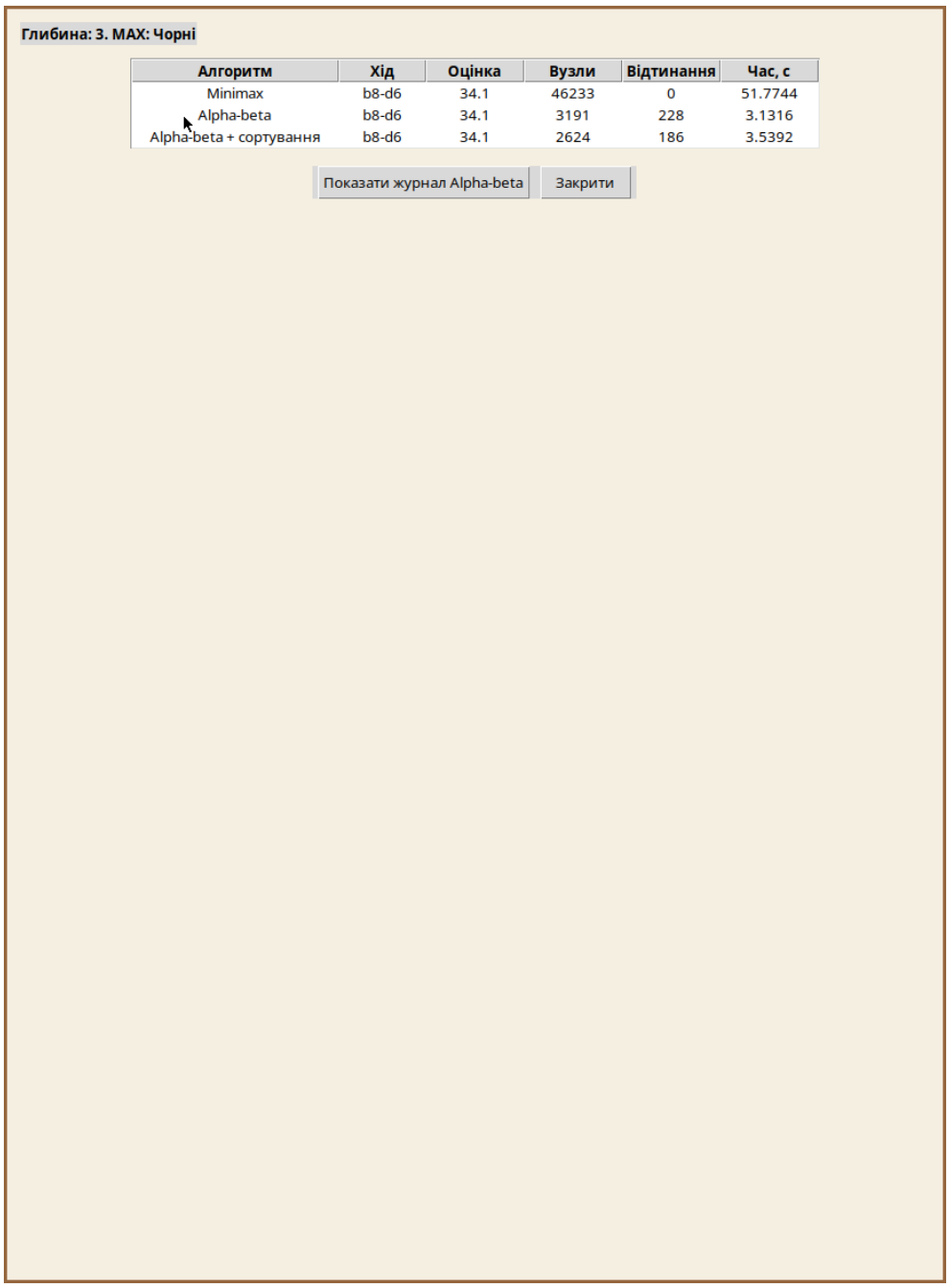


Рисунок 3 – Інтерфейс LOA у процесі гри та статистика ходу Alpha-beta

- Пошук має експоненційну складність; повний Minimax уже на глибині 3 потребував понад 51 с.
- Велика зв’язна компонента не завжди означає тактичну перевагу, якщо її легко розділити наступним ходом.
- Компактність може конфліктувати з мобільністю: надмірне скупчення іноді зменшує кількість ходів.
- Фіксовані ваги HEF підібрані експертно та не є гарантовано оптимальними для всіх стадій гри.
- Ефект горизонту залишається: алгоритм не бачить подій за встановленою глибиною.

11. Ключові фрагменти коду

11.1. Генерація допустимих ходів

```
        for r, c in self.pieces(player):
    for dr, dc in DIRECTIONS:
        distance = self.line_count(r, c, dr, dc)
        er, ec = r + dr * distance, c + dc * distance
        if inside(er, ec) and target_is_not_friendly:
            if no_enemy_piece_on_path:
                moves.append(Move(r, c, er, ec))
```

11.2. Перевірка зв’язності

```
        remaining = set(self.pieces(player))
while remaining:
    stack = [remaining.pop()]
    while stack:
        r, c = stack.pop()
        for dr, dc in DIRECTIONS:
            if (r + dr, c + dc) in remaining:
                remaining.remove((r + dr, c + dc))
                stack.append((r + dr, c + dc))
# Перемога: кількість компонент дорівнює 1
```

```
        if d == 0 or winner != EMPTY or not moves:
    return evaluate(state, max_player), None

maximizing = state.turn == max_player
best = -inf if maximizing else inf
for move in moves:
    value, _ = rec(state.play(move), d - 1, alpha, beta)
    if maximizing:
        best = max(best, value)
        alpha = max(alpha, best)
    else:
        best = min(best, value)
        beta = min(beta, best)
    if alpha >= beta:
        break
```

Висновки

У роботі реалізовано повні правила Lines of Action, графічний інтерфейс, функції UF і HEF, пошук Minimax, Alpha-beta-відтинання та впорядкування ходів. Експеримент підтвердив, що Alpha-beta повертає той самий хід і minimax-оцінку, але різко зменшує обсяг перебору. На глибині 3 кількість вузлів скоротилася з 46 233 до 3 191, а із сортуванням – до 2 624. Найкращий практичний режим для гри – Alpha-beta з обмеженням глибини; доцільність сортування залежить від співвідношення економії вузлів і вартості самої евристики впорядкування.